

Pi Coding Agent: A Practical Primer

24 September 2026

Table of Contents

1	The short version	2
1.1	What you will learn	3
1.2	What this primer is not	3
2	Pi is a harness, not a model	3
3	What Pi provides	4
4	Installation and first run	4
4.1	A better first session	5
4.2	The starter suite: install by need	6
4.3	Hosted authentication	7
4.4	One-shot and structured use	8
5	Hosted and local models	8
5.1	OpenAI-compatible local servers	8
5.2	llama.cpp	8
6	Web research: assemble the missing capability	9
6.1	curl is good for static sources	9
6.2	curl is not web research infrastructure	9
6.3	Start with a maintained search skill	9
6.4	Add a browser only when needed	10
6.5	Search GitHub and forums directly	10
7	Customisation: use the smallest layer that works	11
7.1	Context files and prompt templates	11
7.2	Skills	11
7.3	Extensions	12
7.4	Packages and themes	12
8	Sessions and hand-off	12
9	Safety and containment	13
10	Pi compared with other coding agents	13

10.1 Pi versus OpenCode	14
10.2 Pi versus Aider	14
10.3 Pi versus Claude Code and Codex	15
11 A practical workflow	15
12 Worked example: a reproducible data-science project with Pi and Rocker	15
12.1 Project goal and contract	16
12.2 Give Pi bounded tasks	16
12.3 Build a digest-pinned Rocker image	17
12.4 Run it with rootless Podman	17
12.5 Verify the result independently	18
13 Operational reference	19
13.1 Troubleshooting order	19
13.2 Configuration map	19
13.3 Recommended starting stack	19
14 Reference links	20
15 Closing perspective	20

24 September 2026

1 The short version

Pi is a terminal coding agent and an agent toolkit. It gives a language model a small set of tools, persistent sessions, a terminal UI, and a way to add your own behaviour. Its default tool set is deliberately narrow: read files, write files, edit files, and run shell commands.

That is the key to understanding it. If you expect every coding agent to include a browser, web search, planning mode, subagents, permission prompts, and a large integration catalogue, Pi can look incomplete. If you see the core as a programmable harness, those omissions are the point. You can add a search skill, browser bridge, approval extension, subagent workflow, or local-model provider without forking the main program.

This primer covers installation, first steps, hosted and local models, web and forum research, extensions and packages, sessions, safety, and comparisons with OpenCode, Aider, Claude Code, and Codex. It ends with a worked data-science project using Pi, Podman, and Rocker. Pi changes quickly, so treat provider names and command examples as a snapshot from 24

September 2026. Check `pi --version`, `pi --help`, and the official documentation before applying them.

1.1 What you will learn

1. How Pi's agent loop and extension layers fit together.
2. How to make a controlled first run and build a minimal starter setup.
3. How to use hosted, compatible, and local models.
4. How to give Pi a reliable web and forum research path.
5. When to use a skill, extension, package, or separate service.
6. How to use Pi with Podman and Rocker for a reproducible data-science project.
7. How to preserve, branch, compact, and hand off sessions.
8. When Pi is a better fit than a more opinionated coding agent.

1.2 What this primer is not

This is not a benchmark, a list of every Pi package, or a guarantee that a model, provider, or community extension remains available. It is not a complete TypeScript API reference. The official documentation remains authoritative for exact interfaces and flags.

Pi is not safer than other agents merely because its core is small. It normally runs with the permissions of the launching user. Containment is an operating-system, container, or virtual-machine concern.

The intended reader is comfortable with a terminal, Git, and installing software, but does not yet know Pi's architecture or extension model. You do not need to be a TypeScript developer to use Pi; the extension section assumes only that you can read a small amount of TypeScript when reviewing executable customisation.

2 Pi is a harness, not a model

An AI coding system has four separable parts:

1. **Model:** predicts text and tool calls. It may be hosted or local.
2. **Harness:** builds prompts, presents tools, sends requests, streams output, and records state.
3. **Tools:** give the model effects such as reading files, editing, running commands, searching, or browsing.
4. **Environment:** supplies permissions, network, filesystem, credentials, operating system, and repository.

Pi is mainly the second part, with a useful default tool set and a terminal interface. Its lower-level packages can also be embedded in other applications. Pi is not an inference server and does not make an ordinary model browser-capable. If the model has no search capability and Pi has no search tool, the agent has no dependable web search. It can run curl through bash, but that is an execution primitive, not a research system.

A request follows this shape:

```
your instruction
-> session and system prompt
-> provider and model
-> model response or tool call
-> Pi executes the tool
-> result enters the session
-> model continues
```

The model decides what it wants to do. Pi represents and executes that request. The operating system decides what the process is actually allowed to do.

3 What Pi provides

Pi supplies a terminal UI, provider/model selection, context-file discovery, persistent sessions, and a small default tool set. The commonly documented defaults are read, write, edit, and bash; some versions also expose grep, find, and ls.

That is enough for:

```
inspect repository -> edit file -> run tests -> inspect failure -> edit again
```

Pi does not prescribe a planning mode, subagent system, browser, or general web-search provider in its core. These are workflow choices. You can install a package, write an extension, create a skill, or keep the capability outside Pi and feed the results into the session.

Pi can run interactively, print one-shot output, emit JSON events, communicate in RPC mode, or be embedded through its TypeScript SDK. It is therefore both a person-facing terminal program and a component that can sit inside a larger application.

4 Installation and first run

Pi is distributed as a Node.js package and has an installer. The package and repository names have changed across release generations, so verify the current quickstart. The current documentation uses patterns like:

```
# Prefer downloading and inspecting the installer first. Run it only after
# you understand what it changes.
curl -fsSL https://pi.dev/install.sh -o /tmp/pi-install.sh
less /tmp/pi-install.sh
sh /tmp/pi-install.sh
npm install -g --ignore-scripts @earendil-works/pi-coding-agent
```

```
pi --version
pi --help
```

Older community examples may use @mariozechner/pi-coding-agent or the badlogic/pi-mono URL. Do not mix commands without checking current package metadata. The repository currently redirects through the earendil-works/pi location.

Start in a disposable directory:

```
mkdir -p ~/tmp/pi-first-run
cd ~/tmp/pi-first-run
git init
printf '# Pi test project\n' > README.md
pi
```

Use a read-only first request:

Inspect this repository. Do not edit files or run state-changing commands.
Report the files, likely project type, and three useful next checks.

Before using a real repository, establish which directory Pi considers the project, which instruction files it loaded, which model it selected, and whether it can complete a read-only task.

4.1 A better first session

The first session should prove the complete control loop, not demonstrate how much work Pi can do unattended. Use this sequence:

1. **Verify the executable.** Run `pi --version` and `pi --help`. Record the installed version in your setup notes.
2. **Choose one model.** Start with one tool-capable hosted model or one already-tested local model. Do not add model routing and fallback logic before the basic loop works.
3. **Authenticate explicitly.** Use `/login` or one environment variable. Check that the selected provider and model are the ones you intended.
4. **Orient read-only.** Ask Pi to describe the repository, instructions, checks, and likely change boundaries. If your version supports tool selection, use only read-oriented tools; otherwise state the boundary and verify the diff afterward.
5. **Make one supervised change.** Choose a small documentation or test change. Ask Pi to explain its plan, edit only the named files, run one check, and stop.
6. **Inspect independently.** Run `git status`, `git diff`, and the project's check command yourself. Keep or revert the change based on evidence, not on Pi's confidence.

This sequence exposes the important boundaries early: model authentication, context discovery, tool permissions, repository state, and verification. It also gives you a useful session to resume or discard.

4.2 The starter suite: install by need

There is no universally correct Pi add-on bundle. The project’s design makes a large day-one installation counterproductive: every executable extension adds code to review, configuration to maintain, and another source of prompt injection or data leakage. Start with the smallest stack that supports the work in front of you.

Need	Start with	Add later when	Why it belongs there
Repository conventions	A concise AGENTS.md and the core Pi tools	You repeat a procedure across repositories	Instructions are easier to audit than executable customisation
Reusable procedure	A project or user skill	The procedure needs a model-callable tool or lifecycle hook	Skills package instructions and scripts without changing the agent loop
Public web research	Maintainer pi-skills, especially brave-search	You need another search provider or richer extraction	A narrow search skill is easier to source and constrain than a large bundle
JavaScript or authenticated pages	pi-skills browser-tools with a disposable Chrome profile	You need screenshots, frontend QA, or authenticated forum access	Browser automation is powerful but expands credential exposure
Repeated review prompt	A prompt template	The workflow needs state, events, or custom UI	Templates standardise text without adding executable code
Appearance	Built-in theme	You have a stable daily workflow	Cosmetic customisation should not obscure setup debugging
New model provider or approval gate	A reviewed TypeScript extension	The capability cannot be expressed as a script or skill	Extensions can change tools and lifecycle behaviour

Need	Start with	Add later when	Why it belongs there
Broad third-party web integration	Only after a concrete gap is demonstrated	You need its exact provider mix and accept its data path	Broad packages are convenient but harder to audit

The maintainer-provided skill collection is the sensible first add-on for research. Install the repository once, then install dependencies only for the skill you actually use:

```
git clone https://github.com/badlogic/pi-skills ~/.agents/skills/pi-skills
cd ~/.agents/skills/pi-skills/brave-search && npm install
# Only if interactive browser access is required:
cd ~/.agents/skills/pi-skills/browser-tools && npm install
```

The optional third-party package pi-web-access is listed in the Pi package catalogue and combines search, fetching, extraction, and other web capabilities. It is a community package, not a Pi-maintainer component. Treat it as a candidate to inspect, not as the default recommendation: understand which providers it calls, whether it uses a second answer model, what credentials it stores, and whether its browser features can access cookies. Pin the package and test it in a disposable project if you adopt it.

Do not install an MCP adapter, subagent framework, persistent-memory system, planning overlay, or large “everything” bundle on day one. Add those only when you can name the recurring problem they solve and the boundary they require. A useful rule is: core Pi first, a skill for a repeatable procedure, an extension for a new executable capability, and a package only when you want to distribute a reviewed combination.

4.3 Hosted authentication

You can supply an API key or use an interactive provider login:

```
export ANTHROPIC_API_KEY='session-scoped-secret'
pi
/login
/model
```

Keep secrets out of settings files, prompts, skills, committed scripts, and session notes. Environment variables suit ephemeral runs. Stored authentication is convenient on a workstation but must be protected like any other credential store.

Pi’s live catalogue has included Anthropic, OpenAI, Google, Azure OpenAI, Vertex, Bedrock, OpenRouter, Mistral, Groq, xAI, DeepSeek, Cloudflare, GitHub Copilot, and others. Model IDs, prices, subscription entitlements, and tool support change. Use /model and current provider documentation rather than copying an old model ID.

4.4 One-shot and structured use

```
pi --print 'Summarize this repository without editing it'
git diff --no-ext-diff | pi --print 'Review this diff. Return findings only.'
pi --no-session --print 'Explain this directory'
pi --mode json 'Inspect the repository and report its test commands' > pi-events.jsonl
pi --mode rpc
```

For read-only review, restrict the available tools if your version supports it:

```
pi --tools read,grep,find,ls --print 'Review without changing files'
```

Flags are version-sensitive. Confirm them with `pi -help`.

5 Hosted and local models

Provider flexibility is not the same as model interchangeability. Tool calling, streaming, context length, authentication, image support, and reasoning controls vary. A model that answers chat questions may still be a poor coding-agent model if it produces unreliable tool calls.

5.1 OpenAI-compatible local servers

Ollama, LM Studio, vLLM, and SGLang can expose compatible endpoints. A representative Pi configuration is:

```
{
  "providers": {
    "ollama": {
      "baseUrl": "http://127.0.0.1:11434/v1",
      "api": "openai-completions",
      "apiKey": "ollama",
      "models": [{ "id": "qwen2.5-coder:7b" }]
    }
  }
}
```

The schema changes; consult the current models documentation. A useful arrangement is a local model for routine exploration and private code, with a hosted model as an explicit fallback for difficult debugging or long-context synthesis.

A local model does not make all data local. A search extension, browser, provider gateway, package telemetry, or external API can still receive content.

5.2 llama.cpp

Pi also documents llama.cpp integration. A representative server command is:

```
llama-server \
  --models-dir ~/models \
  --no-models-autoload \
  --jinja \
  --host 127.0.0.1 \
  --port 8080 \
```

```
-ngl 999 \  
-c 32768
```

The current Pi configuration supplies the endpoint and model selection. The Jinja chat template matters for reliable tool calling.

Test the layers in order:

1. Does the inference server answer a health check?
2. Can it answer ordinary chat?
3. Can it stream?
4. Can it produce Pi-compatible tool calls?
5. Can Pi execute a tool and feed the result back?

Keep one known-good hosted fallback while you stabilise the local path.

6 Web research: assemble the missing capability

The concern about Pi’s lack of a browser is well-founded. curl is useful but is not a browser, search engine, or evidence-management system.

6.1 curl is good for static sources

```
curl -fsSL https://raw.githubusercontent.com/earendil-works/pi/main/packages/coding-agent/docs/extensions.md  
curl -fsSL https://api.github.com/repos/earendil-works/pi/releases/latest  
git clone --depth 1 https://github.com/earendil-works/pi.git /tmp/pi-source
```

This is excellent for raw Markdown, JSON APIs, release metadata, source trees, and reproducible downloads.

6.2 curl is not web research infrastructure

A plain HTTP request does not automatically search the web, render JavaScript, preserve signed-in sessions, execute client-side navigation, pass interactive consent, or extract readable text from a complex page. A successful HTTP status can still return only an application shell.

Do not ask Pi to “search the web” and accept model-memory guesses or arbitrary snippets. Give it an explicit tool, source policy, evidence format, and stop condition.

6.3 Start with a maintained search skill

The maintainer’s pi-skills repository has included Brave Search, browser tools, Google-related integrations, YouTube transcripts, and other skills. A representative installation is:

```
git clone https://github.com/badlogic/pi-skills ~/.agents/skills/pi-skills
cd ~/.agents/skills/pi-skills/brave-search
npm install
export BRAVE_API_KEY='session-scoped-secret'
```

Invoke it explicitly:
/skill:brave-search

Research the current Pi extension API. Search official documentation first, then the repository, issues, discussions, and release notes. For each important claim record URL, date, source type, and confidence. Do not install packages or modify files. Return a source table before the synthesis.

The documented helper pattern has looked like:
~/.agents/skills/pi-skills/brave-search/search.js \
'Pi extension registerTool' -n 10 --content --freshness pm

~/.agents/skills/pi-skills/brave-search/content.js \
https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/extensions.md

Check the current skill README for flags. Keep the API key in the environment, not in the skill.

6.4 Add a browser only when needed

For JavaScript-heavy documentation, authenticated forums, or frontend verification, the browser tooling in pi-skills drives Chrome through the Chrome DevTools Protocol, commonly on port 9222, and extracts content after page load.

A browser profile may contain active cookies, tokens, personal history, and private forum access. Use a disposable profile and a narrow account. Do not give an agent your normal browser profile. The browser process, Pi extensions, and shell tools must be treated as mutually trusted code.

Use this decision rule:

Need	Appropriate first tool
Static documentation, JSON, raw GitHub	curl or a narrow shell skill
Search-engine results	Search API skill
JavaScript-rendered public page	Browser or extraction service
Authenticated forum	Disposable browser profile and explicit scope
Frontend testing	Browser automation with screenshots and logs

6.5 Search GitHub and forums directly

```
gh search code 'registerTool repo:earendil-works/pi path:packages/coding-agent'
gh search issues 'MCP repo:earendil-works/pi'
gh api repos/earendil-works/pi/discussions --paginate
```

Search results are leads, not evidence. Read the issue, inspect date and status, and distinguish a maintainer statement from a user workaround.

A useful research brief specifies:

1. The decision the research must support.
2. Source order: official docs/source, release notes, issues, then community material.
3. Required freshness.
4. Evidence format: URL, date, claim, confidence, and support.
5. Stop condition, such as ten sources or three independent confirmations.

For example:

Determine whether Pi currently has a built-in MCP client.
Search the official repository and docs first, then issues and the package catalogue. Do not infer core support from an extension. Return URL, date, source type, exact claim, and confidence. Stop after the current README, docs index, source tree, and five relevant issues or package entries.

This prevents confusing a community package with a core feature or treating an old forum answer as current product behaviour.

7 Customisation: use the smallest layer that works

7.1 Context files and prompt templates

AGENTS.md and CLAUDE.md are appropriate for durable repository guidance: test commands, style, directory boundaries, and review requirements. Prompt templates standardise recurring requests without adding executable behaviour. Keep credentials and volatile provider details elsewhere.

7.2 Skills

A skill is on-demand instructions with optional scripts, references, and assets. It suits a repeatable research procedure or repository QA workflow. Pi can load the full instructions when needed rather than injecting every procedure into every prompt.

A project skill might look like:

```
.pi/skills/web-research/  
  SKILL.md  
  scripts/search.sh  
  references/source-policy.md
```

A skill should remain a procedure. If it needs a model-callable executable tool, it is probably becoming an extension.

7.3 Extensions

Extensions are TypeScript modules executed inside Pi. They can register tools, slash commands, lifecycle handlers, custom UI, confirmation gates, provider adapters, and context or compaction behaviour.

```
pi --extension ./my-extension.ts
```

Typical discovery paths are:

```
~/.pi/agent/extensions/      # global  
.pi/extensions/              # project-local
```

The conceptual shape of a custom tool is:

```
import type { ExtensionAPI } from '@earendil-works/pi-coding-agent';  
  
export default function (pi: ExtensionAPI) {  
  pi.registerTool({  
    name: 'search_release_notes',  
    description: 'Search a bounded local release-note corpus.',  
    parameters: { /* use the current TypeBox schema API */ },  
    execute: async (_toolCallId, params) => {  
      // Validate params, perform a bounded operation, return structured text.  
      return { content: [{ type: 'text', text: String(params) }] };  
    },  
  });  
}
```

An extension is executable code in the Pi process. It can read credentials, alter prompts, intercept tool calls, and start child processes. Review it like application code.

7.4 Packages and themes

Packages combine extensions, skills, prompt templates, and themes for npm or Git distribution. The package catalogue is useful for discovery, not a security review.

```
pi install npm:vendor/package@1.2.3  
pi install git:github.com/vendor/repo@<reviewed-commit>
```

Pin versions or commits. Read source, installation scripts, permissions, network calls, and update history. Test third-party packages in a disposable project. A package that adds browsing, subagents, or unattended execution expands the risk surface quickly.

8 Sessions and hand-off

Pi sessions are persistent conversation records. The active conversation is a branch in a session tree. Pi supports continuing, resuming, branching, forking, cloning, and compaction; exact commands and shortcuts are version-sensitive.

```
pi --continue  
pi --resume  
pi --no-session
```

Branch before a risky experiment. Compact when stale tool output is dominating context, but inspect the summary; compaction is not a perfect archive.

Sessions may contain source code, prompts, command output, URLs, tool arguments, and accidental secrets. Treat them as sensitive data. For a durable hand-off, write:

```
objective
files changed
commands run and results
open questions
assumptions
next safe step
```

9 Safety and containment

Pi is not a sandbox and has no built-in permission boundary for filesystem, process, network, or credential access. It runs with the launching user's permissions. Bash, extensions, package installers, language servers, and child processes can affect whatever that user can affect.

Project trust and containment are different:

- Trust controls whether project-local resources load.
- Tool restrictions control which Pi tools are exposed.
- The operating system, container, VM, or sandbox controls what the process can touch.

For untrusted repositories, unattended runs, production credentials, or browser profiles, use a container or VM. A container is only as strong as its mounts, environment variables, network access, and user identity.

```
docker run --rm -it \
  -e ANTHROPIC_API_KEY \
  -v "$PWD:/workspace" \
  -w /workspace \
  -v pi-agent-home:/root/.pi/agent \
  pi-sandbox
```

Do not mount the whole host home directory or a real browser profile. Do not pass host authentication state unless intended. Give research extensions only the keys and network routes they require.

Prompt injection can come from a README, issue, web page, command output, or model response. Treat external text as untrusted input and review external requests and diffs.

10 Pi compared with other coding agents

The useful comparison is not a feature checklist. Ask where the product puts workflow decisions and what the default execution boundary is.

Agent	Centre of gravity	Local/provider posture	Web posture	Customisation
Pi	Small terminal harness and embeddable runtime	Broad providers plus compatible and local endpoints	Add a skill, extension, package, or shell/API workflow	Deep TypeScript extensions; minimal core
OpenCode	Ready-made terminal coding application	Broad provider and local support	More built-in/configured integrations	Configuration, plugins, skills, and MCP-oriented tools
Aider	Git-aware pair programmer	Many providers and local endpoints	Usually external to the core edit loop	Configuration and scripts; narrower runtime
Claude Code	Opinionated terminal agent around Claude	Strong Anthropic alignment and supported cloud deployments	First-party workflow and integrations	Skills, hooks, MCP, plugins, settings
Codex	Product-connected coding agent	Strong OpenAI alignment and controlled execution	Depends on current product/tool availability	Instructions, skills, plugins, MCP, product workflows

10.1 Pi versus OpenCode

OpenCode feels like a complete coding application. Pi starts smaller and asks you to assemble the workflow. Choose OpenCode when you want a richer experience on day one. Choose Pi when the agent loop itself must be programmable, when you want a narrow local-first harness, or when you want to build an embedded agent.

10.2 Pi versus Aider

Aider's centre is the repository, selected editable files, a repository map, a reviewable edit loop, and Git. Pi can reproduce that loop but can also become a research tool, browser bridge,

custom provider client, or application component. Choose Aider for focused supervised edits; choose Pi when the tool surface and runtime architecture matter as much as the edit.

10.3 Pi versus Claude Code and Codex

Claude Code and Codex supply more surrounding workflow themselves, including established instruction systems and execution controls. Pi gives more responsibility to you: select the model, assemble research access, review extensions, and define containment. That flexibility is not automatically cheaper or safer. Local models can add hardware and quality costs; extensions can add exactly the capability you want while gaining full process access.

11 A practical workflow

1. **Map the repository.** Use a read-only prompt to identify instructions, build commands, tests, generated files, and the smallest safe change.
2. **Research outside it.** Use a search skill or API. Require dated sources and separate facts from inferences.
3. **Hand off evidence.** Write a short reviewed research note with URLs, dates, excerpts, and unresolved uncertainty.
4. **Implement in a branch or worktree.** Give Pi one bounded change, explicit file ownership, and proof commands.
5. **Verify independently.** Run formatter, tests, static checks, and rendering outside the model's narrative. Review the diff and working-tree state.

This keeps research context from overwhelming implementation context and makes the result reproducible.

12 Worked example: a reproducible data-science project with Pi and Rocker

The following project makes the boundary between an agent and an analysis runtime concrete. Pi runs on the host and helps inspect the repository, research package APIs, write code, and interpret failures. Rocker runs the R and Python analysis in a controlled, reproducible container. Podman supplies the rootless container boundary. The Rocker container is not automatically a sandbox for the Pi process; Pi remains a host process unless you separately containerise it.

12.1 Project goal and contract

Build a small “daily temperature quality” project. The input is a fictional CSV of station readings containing dates, station IDs, temperatures, and quality flags. The project must:

- parse ISO dates;
- reject unknown quality flags;
- discard rows marked missing;
- group valid values by date and station;
- emit `n_valid`, `mean_temp_c`, `min_temp_c`, and `max_temp_c`;
- sort the output deterministically; and
- produce the same canonical CSV from independent R and Python implementations.

The repository can be laid out like this:

```
pi-temperature-starter/  
  data/raw/station_readings.csv  
  data/expected/daily_summary.csv  
  r/clean.R  
  r/testthat/test-clean.R  
  python/clean.py  
  python/tests/test_clean.py  
  scripts/compare_outputs.py  
  reports/report.R  
  outputs/  
  renv.lock  
  requirements.txt  
  Containerfile  
  .containerignore  
  Justfile  
  README.md
```

Start with a deliberately tiny fixture:

```
date,station_id,temp_c,quality_flag  
2026-01-01,A,5.1,ok  
2026-01-01,A,,missing  
2026-01-01,B,3.4,ok
```

The expected output is part of the contract, not something Pi is allowed to invent after seeing its own result. The independent implementations should agree on both schema and numeric values. Do not use `reticulate` in this first example: independent R and Python implementations make cross-language agreement visible.

12.2 Give Pi bounded tasks

Use separate sessions or clear hand-offs for research, implementation, and verification.

Prompts like these keep the work auditable:

```
Inspect the repository rules and propose the file contract for this project.  
Do not write files or install packages. Identify assumptions and test cases.  
Research the current official documentation for readr, dplyr, testthat,  
Python csv handling, and pytest. Return URLs and version constraints only.  
Do not modify the repository.
```

Implement the R and Python cleaners from the checked-in CSV contract. Add tests for missing values, unknown flags, malformed dates, empty groups, and cross-language agreement. Use the prescribed container commands and stop if the contract is ambiguous.

The model can draft the code. The fixture, expected output, tests, container build, and independent comparison decide whether the result is correct.

12.3 Build a digest-pinned Rocker image

Use a versioned Rocker image rather than a floating latest tag. The digest below is an example captured for this draft; verify the current digest before using it:

```
ARG ROCKER_IMAGE=ghcr.io/rocker-org/tidyverse:4.6.1
ARG ROCKER_DIGEST=sha256:07711a9ebaceebd70e74957b574c460bb01266881821ff0add5b0b705a1d2ba7
FROM ${ROCKER_IMAGE}@${ROCKER_DIGEST}
```

```
USER root
RUN apt-get update \
  && apt-get install -y --no-install-recommends python3 python3-venv \
  && rm -rf /var/lib/apt/lists/* \
  && python3 -m venv /opt/venv
```

```
ENV PATH="/opt/venv/bin:${PATH}"
WORKDIR /work
```

```
COPY requirements.txt .
# --require-hashes means every requirement must be pinned with a hash.
RUN pip install --no-cache-dir --require-hashes -r requirements.txt
```

```
COPY renv.lock .
RUN R -q -e 'install.packages("renv")' \
  && R -q -e 'renv::restore(prompt = FALSE)' \
  && rm -rf /tmp/downloaded_packages
```

```
USER 1000
```

Create or update `renv.lock` deliberately. The initial dependency installation needs network access; the repeatable test and analysis runs should use an image that has already been built. Pin Python packages in `requirements.txt` with hashes, as required by the command above. Keep `.containerignore` narrow so the build context does not include sessions, credentials, outputs, or unrelated repository data.

12.4 Run it with rootless Podman

A small Justfile can expose the workflow without hiding the container boundary:

```
set shell := ["bash", "-ceu"]
```

```
IMAGE := "localhost/pi-temperature:dev"
OCI := `command -v podman >/dev/null 2>&1 && echo podman || echo docker`
PODMAN_FLAGS := "--userns=keep-id --cap-drop=all"
SECURITY_FLAGS := "--security-opt=no-new-privileges --read-only"
RUNTIME_FLAGS := "--tmpfs /tmp:rw,size=256m --network=none"
```

```

SOURCE_MOUNT := if OCI == "podman" { ":ro,Z" } else { ":ro" }
OUTPUT_MOUNT := if OCI == "podman" { ":Z" } else { "" }

build:
    {{OCI}} build -t {{IMAGE}} -f Containerfile .

test-r:
    {{OCI}} run --rm {{PODMAN_FLAGS}} {{SECURITY_FLAGS}} {{RUNTIME_FLAGS}} \
        -v "{{justfile_directory()}}:/work{{SOURCE_MOUNT}}" \
        -w /work {{IMAGE}} \
        Rscript -e 'testthat::test_dir("r/testthat")'

test-python:
    {{OCI}} run --rm {{PODMAN_FLAGS}} {{SECURITY_FLAGS}} {{RUNTIME_FLAGS}} \
        -v "{{justfile_directory()}}:/work{{SOURCE_MOUNT}}" \
        -w /work {{IMAGE}} \
        pytest -q python/tests

pipeline:
    {{OCI}} run --rm {{PODMAN_FLAGS}} {{SECURITY_FLAGS}} {{RUNTIME_FLAGS}} \
        -v "{{justfile_directory()}}:/work{{SOURCE_MOUNT}}" \
        -v "{{justfile_directory()}}/outputs:/work/outputs{{OUTPUT_MOUNT}}" \
        -w /work {{IMAGE}} Rscript r/clean.R
    {{OCI}} run --rm {{PODMAN_FLAGS}} {{SECURITY_FLAGS}} {{RUNTIME_FLAGS}} \
        -v "{{justfile_directory()}}:/work{{SOURCE_MOUNT}}" \
        -v "{{justfile_directory()}}/outputs:/work/outputs{{OUTPUT_MOUNT}}" \
        -w /work {{IMAGE}} python python/clean.py
    {{OCI}} run --rm {{PODMAN_FLAGS}} {{SECURITY_FLAGS}} {{RUNTIME_FLAGS}} \
        -v "{{justfile_directory()}}:/work{{SOURCE_MOUNT}}" \
        -v "{{justfile_directory()}}/outputs:/work/outputs{{OUTPUT_MOUNT}}" \
        -w /work {{IMAGE}} python scripts/compare_outputs.py

qa: test-r test-python pipeline

```

The exact recipes need adjustment once the project's scripts and output mounts exist. If the pipeline writes outputs while the container filesystem is read-only, mount only the output directory as writable:

```

-v "$PWD/data/raw:/work/data/raw:ro,Z"
-v "$PWD/outputs:/work/outputs:Z"

```

On a system without SELinux relabelling, use the mount syntax appropriate to that runtime. Keep rootless user mapping and explicit mounts; never mount the repository parent, home directory, SSH agent, cloud credentials, container socket, or Podman socket.

12.5 Verify the result independently

Pi should run just qa and report its complete output, but the acceptance decision remains outside the model's prose. Check:

1. Both test suites pass.
2. Unknown flags and malformed dates fail as expected.
3. The missing-value row does not contribute to n_valid or the summary statistics.

4. The R and Python outputs have the same header, sorted keys, row count, and numeric values within the declared tolerance.
5. The report and plot exist under outputs and are not silently written into the source tree.
6. git diff contains only the intended source, lock, fixture, and configuration changes.

If the project later uses real data, keep it outside Git and mount it read-only. Do not give Pi access to sensitive datasets or credentials merely because the analysis container can technically read them. This example teaches reproducibility and review; it does not turn Pi into a trusted data-processing authority.

13 Operational reference

13.1 Troubleshooting order

1. Executable: command -v pi, pi --version, pi --help.
2. Working directory and discovered context files.
3. Provider authentication, without printing secrets.
4. Model response and tool-call support.
5. Provider or search network path.
6. Underlying curl, gh, browser helper, or local server.
7. Extension or skill disabled and retested.
8. Fresh or no-session run to exclude stale context.
9. Independent reproduction of the repository failure.

Do not install another extension until you know which layer failed.

13.2 Configuration map

~/.pi/agent/	user-wide state, auth, settings, sessions
~/.pi/agent/models.json	custom providers and models
~/.agents/skills/	user-wide skills (portable Agent Skills location)
~/.pi/agent/extensions/	user-wide extensions
.pi/	project-local settings and resources
.agents/skills/	project-local skills (portable Agent Skills location)
.pi/extensions/	project-local extensions

13.3 Recommended starting stack

1. Current Pi installer or npm package.
2. One hosted tool-capable provider as fallback.
3. One local OpenAI-compatible server, tested separately.
4. One small project skill for repository procedures.

5. Maintained search skill with a dedicated API key.
6. Browser tooling only when static search is insufficient.
7. Container or disposable worktree for untrusted or unattended work.

Add one capability at a time, test it, record its permissions, and keep it only if it solves a recurring problem.

14 Reference links

- [Pi repository](#)
- [Pi documentation](#)
- [Pi package catalogue](#)
- [Pi model catalogue](#)
- [Pi skills](#)
- [Pi extensions](#)
- [Pi packages](#)
- [Pi prompt templates and themes](#)
- [GitHub code search syntax](#)
- [Podman run reference and Podman build reference](#)
- [Rocker versioned images, extending Rocker images, and rocker-versioned2](#)
- [Aider](#)
- [OpenCode](#)
- [Claude Code](#)
- [OpenAI Codex](#)
- [Claude Code alternatives primer](#)
- [Codex for Claude Code users primer](#)

15 Closing perspective

Pi's distinctive feature is not that it has no browser. It is that the browser, search engine, approval system, model router, planning workflow, and subagent strategy are not all fused into the core. Pi gives you a small, inspectable loop and lets you decide what belongs around it.

Pi works well as a local-first tool, provider-neutral harness, embedded runtime, or inspectable research workflow. It works less well when you want a polished application with integrations ready on day one. Start small, add research access explicitly, isolate the process when necessary, and review every extension as executable code.